CREATORFLOW

MVP Product Requirements

A simple, developer-friendly blueprint for the first working demo

Product goal

Build a small, working content planning app where a creator can sign up, create content drafts, schedule demo posts on a calendar, and view clearly labelled sample analytics. The first version is a demonstration product—not a production social-media publishing platform.

Who this document is for

Developer: Vidya G. Use this document as the agreed MVP scope. If a requirement is unclear or seems too large, ask before adding complexity.

Project approach

Build the simplest reliable version first. Use the skills you already have, learn only what is needed, and deliver a working feature at the end of each week.

MVP scope • 4-week plan • acceptance checklist • learning resources

1. Product overview

CreatorFlow is a demo SaaS for individual content creators. It helps them keep content ideas and drafts in one place, choose a target platform, schedule a planned publishing time, and review a simple overview of their content.

Primary user

An individual creator who wants to organize upcoming content. Build for one creator per account; team collaboration is out of scope.

Main user journey

Sign up → log in → complete a basic profile → create a draft → choose a platform → set a date/time → see it on the calendar → review dashboard counts.

Success means

- A new user can register and log in securely.
- A user can create, view, edit, and delete their own content drafts.
- A user can assign a platform and planned publishing date/time.
- Scheduled demo posts appear on a calendar/list and can be edited or cancelled.
- The dashboard shows counts calculated from the database.
- Sample analytics are visibly marked “Demo data”; no fake data is presented as real platform performance.

2. What to build in the MVP

A. Pages

Page	What it must do
Landing page	Explain the product briefly and provide Sign up / Log in buttons.
Sign up / Log in	Validate inputs, show clear errors, and let a user access their own account.
Dashboard	Show total content items, drafts, scheduled posts, and published-demo posts, based on saved records.
Content	List content items; create, view, edit, and delete them.
Create/Edit content	Fields: title, caption/body, target platform, status, planned date/time (when scheduled).
Calendar / Schedule	Show scheduled demo posts by date. A simple month view or weekly/list view is acceptable.
Profile / Settings	Show and edit basic profile details; provide logout.
Analytics demo	Show a simple chart or summary using sample metrics, with a visible “Demo data” label.

B. Content fields and statuses

Field	Requirement
Title	Required; short name for the content.
Caption / body	Text field; can be optional for an idea but should be encouraged for a draft.
Platform	Choose from YouTube, Instagram, LinkedIn, or Other. This is a label only; no account connection is required.
Status	Idea, Draft, Ready, Scheduled, Published (demo), or Cancelled.
Planned date/time	Required when status is Scheduled. Store consistently and display clearly.

C. What “schedule” means in this demo

> **Scheduling is a demo only**
> When a user schedules a post, save the planned date/time and status in the database and display the post on the calendar. **Do not publish** to Instagram, YouTube, or LinkedIn. **Do not imply** that a post has actually been published. A “Published (demo)” status is only a demonstration state.

D. Dashboard counts

- **Total content** = all content records owned by the logged-in user.
- **Drafts** = records with Draft status (you may include Idea separately).
- **Scheduled** = records with Scheduled status.
- **Published demo** = records with Published (demo) status.
- Counts must update after create/edit/delete/status changes.

3. Recommended technical approach

Use the stack the developer can deliver confidently. The suggested stack below aligns with the project owner's learning goals, but discuss any change before development.

Layer	Preferred option	Notes
Frontend	React.js + CSS	If React is new, learn components, props, state, forms, and fetch/API calls before building all pages.
Backend	Node.js + Express	REST endpoints for authentication and content CRUD. Flask is an acceptable alternative if agreed first.
Database	SQLite	Simple local database for the demo; use parameterized queries.
Authentication	Secure server-side sessions or JWT	Hash passwords with bcrypt; never store plaintext passwords.
Charts	Simple CSS/Chart library	Only one small chart is needed. Keep analytics sample data clearly labelled.
Version control	Git + GitHub	Commit working changes regularly; do not commit secrets or .env files.

Suggested database tables

Keep the schema small. Start with users and content_posts. Add creator_profiles only if profile fields need their own table. Each content record must include its owner/user ID.

- **Minimum users fields:** id, name, email (unique), password_hash, created_at.
- **Minimum content_posts fields:** id, user_id, title, body/caption, platform, status, scheduled_at (nullable), created_at, updated_at.

Minimum API behaviour

- **Authentication:** register, login, logout, and current-user/session endpoint.
- **Content:** list the logged-in user's posts; create, update, and delete a post.
- **Validation:** reject missing titles and invalid scheduled dates; return readable error messages.
- **Security:** every content query must be scoped to the authenticated user. User A must never read or modify User B's content.
- **Configuration:** keep secrets in environment variables; provide a .env.example with placeholder values only.

4. Four-week development plan

Plan for a small, demonstrable deliverable each week. If the developer needs more time to learn a skill, tell the project owner early and adjust the plan rather than hiding the delay.

Week	Build tasks	End-of-week demo / acceptance
1 — Setup & UI	Confirm scope and stack; create GitHub repo; set up frontend/backend; build responsive landing page, sign-up/login screens, dashboard layout, navigation, and reusable UI components. Use temporary UI data only.	Project runs locally; main pages are navigable; design is consistent on desktop and mobile; code is pushed to GitHub.
2 — Backend, database & auth	Create SQLite schema; implement registration/login/logout; hash passwords; protect dashboard routes; connect frontend to backend; handle common errors.	A user can register, log in, log out, and return to their own protected dashboard. Passwords are not stored in plain text.
3 — Content CRUD & calendar	Build create/edit/delete/list content; add platform and status fields; save scheduled date/time; show scheduled items in a simple calendar or schedule list; update dashboard counts from database.	A logged-in user can create a draft, schedule it for a future time, see it on the calendar, edit it, cancel it, and see counts update.
4 — Analytics demo, QA & delivery	Add clearly labelled sample analytics and one simple chart; polish empty/loading/error states; test user data separation; fix bugs; write README and setup steps; prepare deployment if feasible.	End-to-end demo works; sample analytics are labelled; core flows are tested; source code and documentation are handed over.

Weekly update format

At the end of each week, share:

(1) what is complete,
(2) a short screen recording or live demo,
(3) GitHub commits,
(4) what is blocked,
(5) what is planned next week.

5. Learning resources if a skill gap appears

Learn only the topic needed for the next task. The resources below are beginner-friendly official documentation or established learning sites.

Skill	What to learn	Resource
React	Components, props, state, forms, effects, and calling APIs.	React Learn: react.dev/learn
Node + Express	Routes, middleware, request/response, JSON APIs, error handling.	Node: nodejs.org/en/learn Express: expressjs.com/en/starter/installing.html
SQLite / SQL	CREATE TABLE, SELECT, INSERT, UPDATE, DELETE, foreign keys, parameterized queries.	SQLite: sqlite.org/docs.html SQLBolt: sqlbolt.com
Authentication security	Password hashing, session/JWT basics, authorization, input validation.	OWASP Authentication Cheat Sheet: cheatsheetseries.owasp.org/cheatsheets/Authentication_Cheat_Sheet.html
Git + GitHub	Commit, push, branches, README, .gitignore.	GitHub Skills: skills.github.com
Charts	Render a simple bar/line chart with a small dataset.	Recharts: recharts.org/en-US/guide

If the developer uses Flask instead of Express, use the official Flask tutorial and keep the same product behaviour and acceptance criteria.
Flask tutorial: flask.palletsprojects.com/en/stable/tutorial/

6. Out of scope for this MVP

Do not build these features in the first four weeks unless the project owner approves a scope change.

- Real publishing to Instagram, YouTube, LinkedIn, or Facebook.
- Real social account OAuth connections or social platform password collection.
- AI caption generation or AI chatbot.
- Automated reminder emails or inactivity workflows.
- Real social media analytics integrations.
- Subscription payments, pricing plans, or billing.
- Team accounts, agency/client workspaces, approvals, or advanced permissions.
- Mobile app, advanced media upload/storage, complex recurring schedules, or enterprise features.

7. Definition of done

The MVP is complete when all critical checks below pass.

Check	Pass condition
Registration & login	User can register, log in, log out, and see helpful validation errors.
Password security	Passwords are hashed; no plaintext passwords or real secrets are committed to GitHub.
Private data	Two test accounts cannot view, edit, or delete each other’s content.
Content CRUD	Create, list, edit, and delete work and changes persist after refresh.
Scheduling demo	A scheduled post is saved and displayed at the correct date/time; no claim of real publishing.
Dashboard	Counts reflect the signed-in user’s saved content.
Analytics	Sample values and charts are labelled as demo data.
Delivery	GitHub source, README, setup steps, schema, and .env.example are provided.

8. Communication and working agreement

- Ask questions before making major assumptions or adding features.
- Show working software weekly; do not wait until the final week to reveal progress.
- Commit code to the agreed GitHub repository regularly.
- Tell the project owner early if a requirement is unclear, blocked, or taking longer than expected.
- Agree on cost, timeline, ownership of source code, and deployment responsibilities before work starts.
- Use synthetic demo data and test credentials only. Do not use real creator credentials or customer data.

9. Future direction (not part of this build)

After this demo is accepted, the product owner may decide whether to add AI onboarding and activation tracking. Possible future work includes a personalized onboarding checklist, activation events, drop-off tracking, inactivity detection, reminders, and an onboarding funnel. Do not build these now unless separately approved.

> **Final instruction to the developer**
>
> Prioritize a simple, reliable, end-to-end demo over a large number of unfinished features. If the agreed stack needs to change, explain why and get approval before switching. The goal is to deliver a working MVP in four weeks, with a review at the end of every week.